SIMON GAME

I. METHOD

1. Requirements

Refer to: https://www.memozor.com/simon-games/simon-game

Tasks:

- Learn about system functions for generating random numbers.
- Display four squares of different colors (IDs 1 to 4) on the Bitmap Display.
- The first round starts with one square randomly highlighted (brighter color) for 1 second before returning to its normal color.
- The player selects the square by pressing the corresponding key on the keypad, keys 1 to 4.
- If the player inputs the key correctly, the second round starts with two squares randomly highlighted.
- If the player inputs the two keys correctly in sequence, the thrid round starts with three squares randomly highlighted, and so on.
- If the player inputs the keys incorrectly, the game ends.

2. Emulation Hardware Configuration

- Bitmap Display:

  Screen size: 256 x 256 pixels.

  Unit width/height: 8 x 8 pixel.

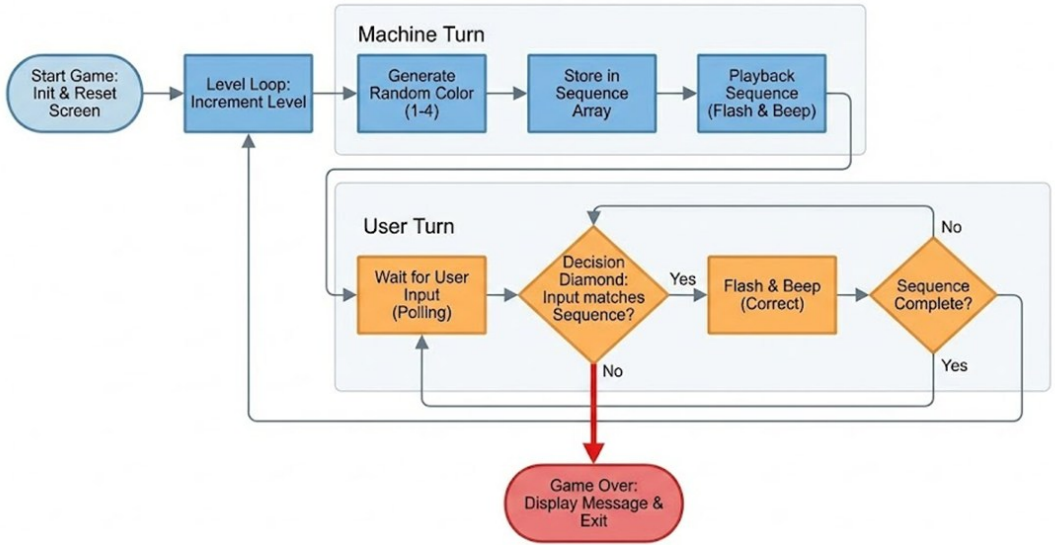
  Base Address: 0x10040000 (Heap).

- Keyboard MMIO (Memory Mapped I/O):

  Use the 0xFFFF0000 address to check the status of the flag.

  Use the 0xFFFF0004 address to read the ASCII code of the key pressed.

II. MAIN ALGORITHMS



The main algorithm is contained in the main game loop. Each iteration represents a Level and consists of 2 main phases:

PHASE 1: MACHINE TURN

Generate a new color, add it to the sequence, and play back the entire sequence for the player to memorize.

1. Generate Random Number:

- Use Syscall 42 to generate an integer in range [0, 3], then add 1 to convert it to range [1, 4].

2. Storage:

- Store the newly generated number into array at index = level - 1.

3. Playback Sequence (machine_run_array):

- Use a loop with counter i from 0 to level - 1.
- Load the color code from array[i].
- Light up the block, play sound (Syscall 31), then delay and turn off block.

PHASE 2: USER TURN

The player must input the exact sequence of colors in the order presented by the machine.

1. Check user input using check_loop that runs while i < level.

- Polling: The program waits here until the user presses a valid key (1, 2, 3, 4). Machine continuously reads address 0xffff0000 (Control receiver). If bit 0 is 1 (key pressed), it reads the ASCII value at 0xffff0004 (Data receiver).
- Comparison:
    - Get user input (t1).
    - Get correct value from array[i] (t2).
    - If Mismatch (t1 != t2): Jump immediately to game_over. Print loss message and terminate.
    - If Match: Call flash.
- Increase i and continue the loop.

2. If the loop finishes successfully (player got the whole sequence right), the program jumps back to Phase 1 to start the next Level.

III. DRAWING LOGIC

To draw a color block (a 14x14-unit block), the algorithm performs the following:

1. Determine block and its ON/OFF status

2. Determine coordinates (x, y):

- Based on input (1, 2, 3, 4), the program sets the starting coordinates (a3, a4).
    - Block 1: (1,1)
    - Block 2: (16,1)
    - Block 3: (1,16)
    - Block 4: (16,16)

3. Drawing:

- Uses two nested loops to iterate through the 14x14 size.

4. Memory Address:

Formula to save (x, y) coordinates in the Heap memory address:

$$Address = Base + (y \times 32 + x) \times 4$$

- *The screen's width is 32 units, the units are stored in row-major order.*
- *1 unit = 1 word = 4 bytes*